

Documentation d'Architecture Compilateur Deca

Équipe GL 51
Ensimag

13 janvier 2026

Version 1.0

Table des matières

1 Introduction

1.1 Contexte et Objectifs

Ce document décrit l'architecture interne du compilateur Deca. Il s'adresse aux développeurs qui maintiendront ou feront évoluer ce compilateur. Le compilateur Deca traduit des programmes Deca (langage similaire à Java) en assembleur IMA. Ce document explique **comment** le compilateur fonctionne, pas comment l'utiliser.

1.2 Qui ? Quoi ? Pourquoi ? Quand ? Pour Qui ?

- **Qui** : L'équipe GL 51, projet Ensimag 2026
- **Quoi** : Un compilateur complet (lexer, parser, analyse contextuelle, génération de code)
- **Pourquoi** : Traduire Deca vers IMA pour exécution sur machine abstraite
- **Quand** : Développé entre décembre 2025 et janvier 2026
- **Pour qui** : Futurs mainteneurs du code, développeurs internes

2 Organisation Générale

2.1 Pipeline de Compilation

Le compilateur suit un pipeline classique en quatre phases :

1. **Analyse lexicale** : Découpe le fichier source en tokens (`DecaLexer.g4`)
2. **Analyse syntaxique** : Construit l'arbre de syntaxe abstraite (`DecaParser.g4`)
3. **Analyse contextuelle** : Vérifie types et portées (méthodes `verify*()`)
4. **Génération de code** : Produit l'assembleur IMA (méthodes `codeGen*()`)

2.2 Orchestration : DecacCompiler

La classe `DecacCompiler` coordonne toutes les phases. Elle contient :

- `symbolTable` : Table des symboles globale (internement de chaînes)
- `environmentType` : Types prédéfinis (`int`, `float`, `boolean`, `void`)
- `program` : Représentation IMA accumulant les instructions
- `registerManager` : Gestionnaire d'allocation de registres

Flux d'exécution :

```
compile() -> doCompile() -> doLexingAndParsing()
           -> verifyProgram() -> codeGenProgram()
```

Chaque méthode délègue au `DecacCompiler` pour ajouter des instructions :

```
1 compiler.addInstruction(new LOAD(...));
2 compiler.addComment("debut boucle");
```

3 Description des Paquetages et Classes Principales

3.1 fr.ensimag.deca.syntax

Contient les grammars ANTLR4 et le code généré.

- DecaLexer.g4 : Règles lexicales (tokens : IF, WHILE, IDENT, etc.)
- DecaParser.g4 : Règles syntaxiques (production AST)
- Classes générées : DecaLexer, DecaParser, listeners

Dépendance : ANTLR4 runtime 4.13.2

3.2 fr.ensimag.deca.tree

Représente l'arbre de syntaxe abstraite (AST). Chaque nœud hérite de Tree.

Hiérarchie principale :

```

Tree (racine)
  AbstractProgram      Program
  AbstractMain         Main, EmptyMain
  AbstractExpr
    AbstractBinaryExpr
      AbstractOpArith   Plus, Minus, Multiply, Divide,
Modulo
      AbstractOpCmp     Equals, NotEquals, Greater,
Lower, etc.
      AbstractOpBool    And, Or
      AbstractUnaryExpr  Not, UnaryMinus
      AbstractLiteral    IntLiteral, FloatLiteral,
BooleanLiteral
      Identifier, ReadInt, ReadFloat
  AbstractInst
    IfThenElse, While
    Assign, Print, Println
    NoOperation

```

Responsabilités :

- Chaque nœud implémente `verifyExpr()` ou `verifyInst()` (phase contextuelle)
- Chaque nœud implémente `codeGenExpr()` ou `codeGenInst()` (phase génération)
- `decompile()` permet de restituer le code source

Dépendances :

- `fr.ensimag.deca.context` pour types et environnements
- `fr.ensimag.ima.pseudocode` pour instructions IMA

3.3 fr.ensimag.deca.context

Gère l'analyse contextuelle (vérification de types et portées).

Classes clés :

- `Type` : Hiérarchie des types (`IntType`, `FloatType`, `BooleanType`, `ClassType`, etc.)
- `EnvironmentType` : Environnement global des types
- `EnvironmentExp` : Environnement local pour variables (structure de pile)

- **Definition** : Définitions liées aux symboles (`VariableDefinition`, `MethodDefinition`, etc.)

Structure de EnvironmentExp :

Liste chaînée de dictionnaires (`Map<Symbol, ExpDefinition>`). Chaque bloc introduit un nouvel environnement fils. La recherche remonte vers le parent si non trouvé.

Mécanisme de décoration :

Lors de `verify*()`, chaque expression se voit attribuer un type via `setType()`. Ces types sont utilisés en génération de code.

3.4 fr.ensimag.deca.codegen

Contient uniquement `RegisterManager` pour l'allocation de registres.

Dépendances externes :

`fr.ensimag.ima.pseudocode` fournit les instructions IMA (`LOAD`, `STORE`, `ADD`, `BEQ`, etc.).

3.5 fr.ensimag.ima.pseudocode

Représentation abstraite de l'assembleur IMA.

- `IMAProgram` : Liste de lignes (instructions + labels + commentaires)
- `Instruction` : Classes comme `LOAD`, `ADD`, `BEQ`, etc.
- `GPRRegister` : Registres généraux R0-R15
- `DAddr` : Adresses (registres, offsets)

4 Algorithmes et Structures de Données Spécifiques

4.1 Allocation de Registres : Stratégie Naïve

Choix : Allocation naïve avec spill automatique sur pile.

Implémentation (`RegisterManager`) :

- Registres R2 à R15 disponibles (R0-R1 réservés scratch)
- Compteur `registreCourant` : prochain registre libre
- Méthodes :
 - `prendreRegistre()` : alloue un registre, échoue si tous utilisés
 - `libererRegistre()` : libère le dernier registre alloué (stack-like)

Stratégie dans AbstractOpArith :

1. Évaluer opérande gauche dans `register`
2. Si registre libre : évaluer opérande droite dans nouveau registre, puis `OP reg_right, register`
3. Si aucun registre libre (spill) :
 - `PUSH register` (sauvegarder gauche sur pile)
 - Évaluer opérande droite dans `register`
 - `LOAD register, R0` puis `POP register`

— OP R0, register

Justification :

- **Simplicité** : Pas de graphe d'interférence, pas d'analyse de vivacité
- **Suffisant pour Deca** : Expressions simples, peu d'imbrications profondes
- **Inconvénient** : Spill fréquent pour expressions complexes (overhead pile)
- **Alternative rejetée** : Allocation par coloration de graphe (complexité excessive pour GL)

4.2 Génération de Code pour le Flot de Contrôle

Stratégie Labels :

Les structures if et while utilisent des labels uniques pour les sauts.

IfThenElse :

```
<Code(Condition, faux, E_Sinon)>
<Code(Alors)>
BRA E_Fin
E_Sinon:
<Code(Sinon)>
E_Fin:
```

- Compteur statique c incrémenté pour labels uniques (else.1, end_if.1, etc.)
- codeGenBool(compiler, false, elseLabel) : saute si condition fausse

While :

```
BRA E_Cond.n
E_Debut.n:
<Code(Corps)>
E_Cond.n:
<Code(Condition, vrai, E_Debut.n)>
```

- Évaluation condition en fin de boucle (évite duplication)
- Saut initial vers condition (cohérence sémantique)

Génération de Conditions :

codeGenBool(compiler, branchOn, targetLabel) évalue une expression booléenne :

1. Évaluer expression dans registre
2. CMP #0, register
3. Si branchOn == true : BNE targetLabel (sauter si vrai)
4. Si branchOn == false : BEQ targetLabel (sauter si faux)

Justification :

- **Labels uniques** : Évite conflits dans boucles imbriquées
- **Évaluation paresseuse non implémentée** : && et || évaluent toujours les deux opérandes (simplification)
- **Alternative** : Court-circuit (complexe, non requis par sujet)

4.3 Gestion de la Pile : TSTO/ADDSP

Stratégie :

RegisterManager trace la taille maximale de pile nécessaire.

Compteurs :

- `taillePileCourante` : Profondeur actuelle de pile
- `taillePileMax` : Maximum atteint (pour TSTO)
- `nbGlobales` : Nombre de variables globales

Méthodes :

- `empiler()` : Incrémente compteurs (lors de PUSH ou spill)
- `depiler()` : Décrémentes (lors de POP)
- `ajouterVariablesLocales(n)` : Réserve espace pour variables locales

Génération en-tête (`Program.codeGenProgram()`) :

Instructions insérées en **ordre inverse** (LIFO via `addFirstInstruction()`) :

1. `TSTO #(maxTemp + nbGlob)` : Vérifie espace disponible
2. `BOV stack_overflow_error` : Saute vers gestion erreur si overflow
3. `ADDSP #nbGlob` : Réserve espace pour variables globales

Gestion d'erreurs :

Labels pour erreurs runtime :

```
stack_overflow_error:
    WSTR "Error: Stack Overflow"
    WNL
    ERROR
```

Justification :

- **Analyse statique** : Calcul conservateur de taille pile (pas de récursion donc borné)
- **Sécurité** : `BOV` empêche corruption mémoire
- **Coût** : Une instruction `TSTO` par programme (négligeable)
- **Alternative rejetée** : Pile dynamique (non supporté par IMA)

4.4 Conversion de Types Implicites

Stratégie :

Si opération arithmétique mélange `int` et `float`, convertir `int` en `float`.

Implémentation (`AbstractOpArith.verifyExpr()`) :

```
1 if (t1.isFloat() && t2.isInt()) {
2     setRightOperand(new ConvFloat(getRightOperand()));
3     // Reverifier l'operande convertie
4 }
```

Node `ConvFloat` génère instruction IMA `FLOAT register, register`.

Justification :

- **Norme Deca** : Promotions implicites `int`→`float` (comme Java)
- **Simplicité AST** : Insertion transparente de nœuds de conversion
- **Alternative** : Conversion runtime (moins efficace, nécessite métadonnées type)

5 Stratégie de Développement

5.1 Approche Incrémentale

Le projet suit un développement par étapes :

1. **Étape A (Lexer/Parser)** : Grammaires ANTLR4, tests lexicaux/syntaxiques
2. **Étape C "Sans Objet"** : Génération de code procédural (sans classes)
3. **Étape B (Analyse Contextuelle)** : Vérification types et portées
4. *Étape C Complète (Non implémentée)* : Support OO (classes, héritage, méthodes virtuelles)

5.2 Stratégie de Tests

Tests par phase :

- **Lexicaux** : `src/test/script/basic-lex.sh + test_lex` launcher
 - Fichiers valides dans `src/test/deca/lexical/valid/`
 - Fichiers invalides dans `src/test/deca/lexical/invalid/`
- **Syntaxiques** : `basic-synt.sh + test_synt`
- **Contextuels** : `basic-context.sh + test_context`
 - Vérifie erreurs de types, variables non déclarées
- **Génération** : `basic-gencode.sh`
 - Compile et exécute avec IMA, vérifie sortie

Tests unitaires JUnit 5 :

Fichiers dans `src/test/java/fr/ensimag/deca/` :

- `TestWhileCodeGen` : Teste génération de boucles
- `TestBooleanOpsCodeGen` : Opérateurs logiques
- `TestPlusPlain`, `TestPlusAdvanced` : Vérification contextuelle

Test de régression :

`./src/test/script/common-tests.sh` : Suite minimale validant fonctionnalités de base (hello world, syntaxe invalide, variables non déclarées).

5.3 Couverture de Code

JaCoCo configuré via Maven (`jacoco.skip=false`). Rapport généré dans `target/site/jacoco/`.

Exécution :

```
mvn clean test -Djacoco.skip=false
./src/test/script/jacoco-report.sh
```

Exclusions : Classes d'erreurs internes (`DecacInternalError`, `IMAInternalError`) exclues de la couverture.

6 Spécifications Techniques

6.1 Contraintes d'Environnement

- **Java** : Version 21 requise (déclaré dans `pom.xml`)
- **Maven** : 3.6.3 minimum (vérifié par `maven-enforcer-plugin`)
- **ANTLR4** : 4.13.2 (dépendance Maven)
- **JUnit** : 5 (Jupiter) pour tests unitaires

6.2 Conventions de Codage

Nommage :

- Classes abstraites préfixées **Abstract** (ex : `AbstractExpr`)
- Méthodes `verify*()` retournent `Type` et lèvent `ContextualError`
- Méthodes `codeGen*()` sont `protected` et ne retournent rien

Gestion d'erreurs :

- **Erreurs lexicales/syntaxiques** : ANTLR rapporte sur `System.err` (format `file:line:message`)
- **Erreurs contextuelles** : `throw new ContextualError(msg, location)`
- **Erreurs fatales** : `throw new DecacFatalError(msg)` (I/O, assertions violées)

Assertions Java :

Le code utilise massivement `assert`. JVM doit être lancée avec `-ea` (activé dans script `decac` et launchers de test).

Pattern Visitor :

Bien que non explicite, l'AST suit un pattern Visitor implicite :

- `verifyExpr()` traverse l'arbre en pré-ordre
- `codeGenExpr()` traverse en post-ordre (évaluation des fils avant le parent)

6.3 Limitations Connues

- **Pas de compilation parallèle** : Flag `-P` lève `UnsupportedOperationException`
- **OOP partiel** : Classes et héritage non complètement implémentés (étape C objet manquante)
- **Pas d'optimisation** : Code généré non optimisé (pas de pliage de constantes, pas d'élimination de code mort)
- **Registres limités** : Spill naïf peut générer beaucoup de `PUSH/POP` pour expressions complexes

6.4 Fichiers de Configuration

- `pom.xml` : Configuration Maven (dépendances, plugins ANTLR4, JaCoCo)
- `src/main/resources/log4j.properties` : Configuration logging
- `src/main/config/findbugs-exclude.xml` : Exclusions analyse statique
- `src/test/script/launchers/*` : Scripts configurant classpath pour tests par phase

7 Conclusion

Cette architecture privilégie la **simplicité et la maintenabilité** :

- Pipeline classique de compilation
- Allocation naïve de registres (sans graphe d'interférence)
- Tests modulaires par phase
- Séparation claire AST / Context / CodeGen

Le compilateur remplit son objectif pédagogique : traduire Deca procédural vers IMA. Les extensions futures (OOP complet, optimisations) nécessiteront :

- Refonte de **RegisterManager** pour allocation globale
- Ajout de passes d'optimisation (DCE, CSE, inlining)
- Support tables de méthodes virtuelles (VMT)

Contacts : Équipe GL 51, Ensimag 2026